# Explanation Report

## Introduction

This report explains the implementation of Graph Data Structures and Dijkstra's Algorithm in a space simulation project. It uses real-world analogies to simplify concepts and prepares you for viva questions.

## Core Concepts

### What is a Graph?

- • Definition: A graph consists of nodes (vertices) and edges connecting them.
- • Analogy: Cities = nodes, Roads = edges, Distance = edge weight.

In the project: planets, particles, and black hole = nodes; distances = weighted edges.

### What is an Adjacency Matrix?

- • Purpose: Represents connections between nodes in a 2D table.
- • Initialization: adj = [[float("inf")]*n for _ in range(n)]
- • Meaning: Start with ∞ (unreachable), update with actual distances later.

## Building the Graph

- • Combine all nodes into a master list for indexing.
- • Connect planets using Euclidean distance if within threshold.
- • Connect particles to their planets using orbit radius.
- • Connect black hole to planets within range.

## Dijkstra's Algorithm

Goal: Find shortest path from a start node to all others.

Steps:

1. 1. Initialize distances as ∞, except start = 0.
2. 2. Use a priority queue (min-heap) for closest node selection.
3. 3. Relax edges: update distance if a shorter path is found.

## Path Reconstruction

Trace back using prev[] array and reverse the path.

# Common Viva Questions

4. Q1: Why Dijkstra instead of BFS?

A: BFS works for unweighted graphs; Dijkstra handles weights.

5. Q2: What if negative weights exist?

A: Use Bellman-Ford; Dijkstra fails with negatives.

6. Q3: Time Complexity?

A: Matrix: $O(V^2)$; List + Heap: $O((V+E) \log V)$.

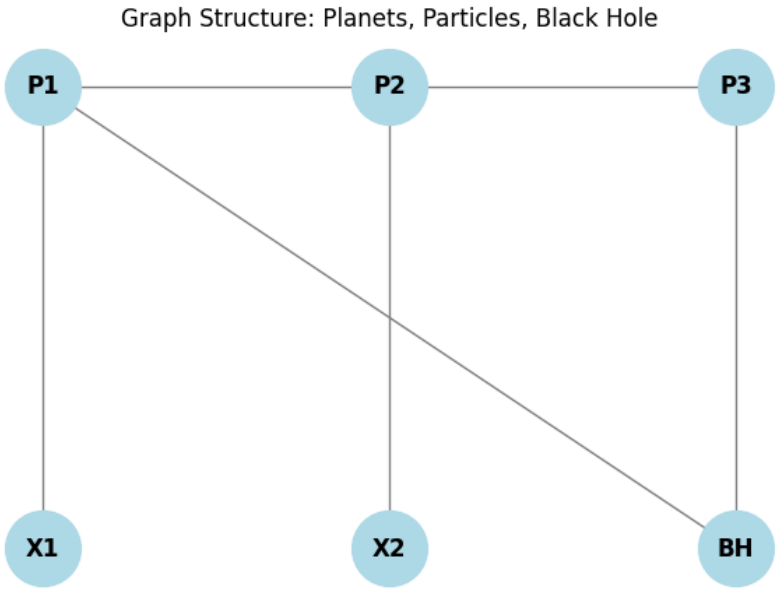
7. Q4: Why rebuild graph every frame?

A: Because planets move; dynamic graph needed.

# Conclusion

This breakdown covers graph representation, adjacency matrix, dynamic edge creation, and Dijkstra's algorithm. Understanding these concepts ensures confidence during viva.

# Diagrams:



| | Ali | Bilal | Sara | Dania |
|---|---|---|---|---|
| Ali | 0 | ✓ | ✓ | ✗ |
| Bilal | ✓ | 0 | ✗ | ✗ |
| Sara | ✓ | ✗ | 0 | ✓ |
| Dania | ✗ | ✗ | ✓ | 0 |

[Dijkstra Flow]

**Dijkstra's Algorithm**:SS

1st**Start:** Initialize dist[], prev[], and priority queue.

2nd **Pick node:** Select the node with the smallest distance.

3rd **Skip outdated entries:** Ignore stale queue entries.

4th **Relax edges:** Update distances and predecessors.

5th **Push updates:** Add updated nodes to the queue.

6th **Repeat:** Continue until the queue is empty.

7th **Reconstruct path:** Use prev[] to trace the shortest path.

Although the objects continuously move due to orbital animation, the underlying Data Structures and Algorithms (DSA) components remain static. The project demonstrates how real-time animations can be layered on top of a fixed graph without altering the graph's logical structure. This section explains how paths are created, how distances are computed, and how Dijkstra's algorithm is applied in the context of moving graphical objects.

# 2. Static Graph Construction

At the beginning of execution, all celestial objects are stored inside a list:

```
objects = [planet1, planet2, particle1, blackhole1, ...]
```

Each object has two essential properties:

**x-coordinate**

**y-coordinate**

These coordinates represent the object's position on the 2D display.

The program uses these initial coordinates to construct a **Graph**, represented via either:

- an **Adjacency Matrix**, or
- 
- an **Adjacency List**
- 

This graph encodes:

- Nodes → objects
- 
- Edges → possible travel paths between objects
- 
- Weights → Euclidean distances between objects
- 

This graph is **static** and created only once.

---

# 3. Distance (Weight) Calculation

To compute the weight of each edge, the program uses **Euclidean Distance**:

d=(x2−x1)2+(y2−y1)2d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}d=(x2−x1)2+(y2−y1)2

This represents the straight-line distance between two objects in 2D space.

## Why Euclidean Distance?

Because the simulation models free movement in a continuous space, not constrained movement on a grid.

## DSA Importance

Edge weight directly affects:

- Shortest path selection
- Priority Queue operations in Dijkstra's algorithm
- Graph optimization

---

# 4. Orbital Movement Is Pure Animation (Graph Stays Static)

Although the objects visually move due to orbital equations such as:

```
self.x = center_x + radius * cos(angle)
```

```
self.y = center_y + radius * sin(angle)
```

this movement does **not** change the graph.

## Key Explanation

- The **graph is built once** using the initial positions.
- **Movement happens only on-screen**, not in the graph structure.
- Dijkstra's algorithm always runs on the original graph.

This creates the appearance of dynamic connections even though the back-end logic is fixed.

---

# 5. How Paths are Formed During Movement

When a user selects two nodes:

1stThe program extracts their **indices** in the adjacency matrix.

2nd It runs **Dijkstra's algorithm** on the static graph.

3rd The algorithm returns a list of nodes representing the **shortest path**.

Example:

```
A → C → F → H
```

## How the path is drawn while objects move

Every frame:

- The program retrieves the *current* (moving) x, y positions of nodes A, C, F, H.
- It draws lines connecting these positions.
- Visually, the path appears dynamic because the objects move.
- Logically, the path is static because the graph is static.

## Real-Life Analogy

This is similar to:

- Calculating a route on Google Maps (static)
- Watching the car move along the road (dynamic)

The road network does not change; only the car moves.

---

# 6. Why This is a Valid DSA Project

This project showcases several core DSA concepts:

## 6.1 Graph Theory

- Node representation
- Edge construction
- Weighted graphs
- Graph traversal

## 6.2 Euclidean Weight Assignment

- Real-world modeling of physical distances
- Geometric computation
- Proper weighting for shortest path algorithms

## 6.3 Dijkstra's Algorithm

- Shortest path computation
- Use of Priority Queue (Min-Heap)
- Relaxation of edges
- Time complexity management

## 6.4 Adjacency Matrix Usage

- Storing and accessing edges in O(1)
- Representing fully connected graphs
- Trade-offs vs adjacency lists

## 6.5 Simulation Layer Over DSA Layer

This project proves a key CS concept:

The visualization layer and the algorithmic layer can operate independently.

# The Complete DSA Breakdown — Line by Line, Like You Know Nothing

Aqi, I'm about to teach you this code as if you're sitting in a Princeton CS lecture. I'll use **real-world analogies**, explain every DSA concept from **absolute zero**, and prep you to **destroy that viva**. By the end, you'll explain this code better than the person who wrote it.

## PART 1: What is a Graph? (The Foundation)

### Real-life analogy:

Imagine you're planning a road trip across Pakistan. You have:

- **Cities** (Lahore, Karachi, Islamabad) = **Nodes/Vertices**
- **Roads connecting them** = **Edges**
- **Distance on each road** = **Edge Weight**

This entire system = **a graph**.

In your project:

- **Planets, particles, black hole** = nodes
- **Distances between them** = weighted edges
- The whole space system = **a weighted graph**

## PART 2: What is an Adjacency Matrix?

An **adjacency matrix** is a 2D table showing connections between nodes.

### Real-life example:

Imagine a WhatsApp group with 4 friends: Ali, Bilal, Sara, Dania.

| | **Ali** | **Bilal** | **Sara** | **Dania** |
|---|---|---|---|---|
| Ali | 0 | | | |
| Bilal | | 0 | | |
| Sara | | | 0 | |

| | |
|---|---|
| Dani<br>a | 0 |

1. = they chat (edge exists)
2. = they don't chat (no edge)
3. 0 = can't chat with yourself

In your code:
adj = [[float('inf')]*n for _ in range(n)]

**Translation:**

- Create an n × n table
- Fill everything with ∞ (infinity) meaning "no road exists"
- Later, replace ∞ with actual distances where roads DO exist

**Why float('inf')?**
Because in Dijkstra, we need to compare distances. ∞ means "unreachable for now". It's like saying "this route is impossible until proven otherwise."

# PART 3: Building the Graph (Line-by-Line)

all_nodes = nodes + particles + [bh]

## What's happening:

You're creating a **master list** of all objects in space.

**Real-life analogy:**
Imagine you're organizing a wedding:

1. nodes = family members (6 people)
2. particles = friends (15 people)
3. [bh] = the groom (1 person)
4. all_nodes = **complete guest list** (22 people total)

**Why this matters:**
Later, when you say "index 0", you mean the first planet. "Index 18" means some particle. This list is the **address book** for your graph.

n = len(all_nodes)

**Simple:** Count how many total objects exist. This determines matrix size.

```
adj = [[float('inf')]*n for _ in range(n)]
```

**Creating the adjacency matrix.**

**Breakdown:**

1. [float('inf')]*n = create a row of n infinities
2. for _ in range(n) = create n such rows
3. Result: an n × n grid filled with ∞

**Memory cost:** O(n²) — for 22 nodes, that's 484 cells. Totally fine.

## Connecting Planets to Each Other

```
for i in range(len(nodes)):
    for j in range(len(nodes)):
        if i != j:
            d = (nodes[i].pos - nodes[j].pos).length()
            if d < 400:
                adj[i][j] = d
```

**Line-by-line dissection:**

### *Line 1-2: Nested loops*

```
for i in range(len(nodes)):
    for j in range(len(nodes)):
```

**What this does:**
Check **every possible pair** of planets.

**Real-life analogy:**
You have 6 friends. You want to see who lives close to whom. You compare:

- Friend 0 with friends 1, 2, 3, 4, 5
- Friend 1 with friends 0, 2, 3, 4, 5
- ...and so on

**DSA concept:** This is a **brute-force pairwise comparison**. Time complexity: **O(P²)** where P = number of planets.

### *Line 3: Avoid self-loops*

```
if i != j:
```

**Why:** A planet can't have a road to itself. Like saying "distance from Lahore to Lahore = 0" — pointless to store.

### *Line 4: Calculate Euclide distance*

```
d = (nodes[i].pos - nodes[j].pos).length()
```

**What's happening:**

- nodes[i].pos = coordinates of planet i (e.g., (200, 300))
- nodes[j].pos = coordinates of planet j (e.g., (500, 700))
- Subtraction gives a **vector** between them
- .length() calculates **Euclidean distance**: $\sqrt{[(x_2-x_1)^2 + (y_2-y_1)^2]}$

**Real-life analogy:**
If Lahore is at (31.5°N, 74.3°E) and Karachi is at (24.8°N, 67.0°E), you calculate the straight-line distance on a map.

**DSA concept:** Distance calculation is **O(1)** — just math.

### *Line 5-6: Proximity threshold*

```
if d < 400:
   adj[i][j] = d
```

**What this means:**
Only create an edge if planets are **within 400 units**.

**Why?**
Because if planets are too far apart, they shouldn't have a direct connection. This makes the graph **sparse** (fewer edges), which speeds up Dijkstra.

**Real-life analogy:**
You only mark roads between cities within 500 km. Lahore-Karachi is 1200 km, so no direct road in your graph — you'd need intermediate cities.

**DSA concept:** This creates a **sparse graph**, which is efficient for pathfinding.

## Connecting Particles to Planets

```
for idx, p in enumerate(particles):
   pi = len(nodes) + idx
   planet_idx = nodes.index(p.planet)
```

```
    adj[pi][planet_idx] = p.orbit_radius
    adj[planet_idx][pi] = p.orbit_radius
```

### *Line 1: Loop through particles*

for idx, p in enumerate(particles):

enumerate gives you both the particle (p) and its position (idx) in the list.

### *Line 2: Find particle's index in master list*

pi = len(nodes) + idx

**Why?**
Particles come **after** planets in all_nodes. So if there are 6 planets:

- Planet indices: 0, 1, 2, 3, 4, 5
- Particle indices: 6, 7, 8, 9, ... 20
- Black hole index: 21

**Real-life analogy:**
At the wedding, family sits in rows 1-6, friends in rows 7-21, groom in row 22.

### *Line 3: Find which planet the particle orbits*

planet_idx = nodes.index(p.planet)

**What this does:**
Search through nodes to find which planet this particle belongs to.

**DSA issue:** nodes.index() is **O(P)** — linear search. For 15 particles, this is 15 × 6 = 90 comparisons per frame.

**Optimization:** Store planet_idx when creating the particle instead of searching every time.

### *Line 4-5: Create bidirectional edge*

adj[pi][planet_idx] = p.orbit_radius
adj[planet_idx][pi] = p.orbit_radius

**What's happening:**

8. Particle → Planet: distance = orbit radius
9. Planet → Particle: same distance (undirected edge)

**Why bidirectional?**
Because you want to allow paths in both directions. It's like a two-way street.

**DSA concept:** Undirected edges require setting **both** adj[i][j] and adj[j][i].

### Connecting Planets to Black Hole

```
for i, node in enumerate(nodes):
    d = (node.pos - bh.pos).length()
    if d < 500:
        adj[i][len(all_nodes)-1] = d
        adj[len(all_nodes)-1][i] = d
```

**Same logic as planets:**

- Calculate distance from each planet to black hole
- If within 500 units, create edge
- len(all_nodes)-1 = black hole's index (last in list)
- Set both directions (undirected)

# PART 4: Dijkstra's Algorithm (The Heart)

```
def dijkstra(adj, start):
    n = len(adj)
    dist = [float('inf')]*n
    prev = [-1]*n
    dist[start] = 0
    pq = [(0, start)]
```

## What is Dijkstra solving?

**Problem:** Find the **shortest path** from one node to all others in a weighted graph.

**Real-life analogy:**
You're at Lahore and want to find the fastest route to every other city in Pakistan. Dijkstra calculates:

- Lahore → Islamabad: 370 km (shortest route)
- Lahore → Karachi: 1200 km (via Multan, not direct)

## Initializations:

```
dist = [float('inf')]*n
```

**Meaning:** "I don't know any distances yet, assume everything is unreachable."

prev = [-1]*n

**Meaning:** "I don't know the previous node in the shortest path yet."

**Why track prev?**
So you can **reconstruct the path** later. Like breadcrumbs showing how you got somewhere.

dist[start] = 0

**Meaning:** "Distance from start to itself = 0."

pq = [(0, start)]

**What's a priority queue?**
A **min-heap** that always gives you the smallest element first.

**Real-life analogy:**
Imagine a hospital ER. Patients are prioritized by severity:

- Critical (priority 1) gets seen first
- Moderate (priority 5) waits
- Minor (priority 10) waits longer

In Dijkstra, the "priority" is **distance**. You always explore the **closest unvisited node** first.

**Python's heapq:**

- heappush: add element (O(log n))
- heappop: remove smallest (O(log n))

## Main Loop:

```
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]:
        continue
```

### *Line 1: While queue isn't empty*

Keep exploring nodes.

### *Line 2: Pop the closest node*

```
d, u = heapq.heappop(pq)
```

**What this means:**
Get the node u with the smallest tentative distance d.

**Real-life analogy:**
You're at Lahore. Your options:

1. Go to Islamabad (370 km)
2. Go to Multan (350 km)

You pick **Multan** because it's closer.

### *Line 3: Skip stale entries*

```
if d > dist[u]:
    continue
```

**Why?**
You might have added node u to the queue multiple times with different distances. If this distance is outdated (you already found a better route), skip it.

**DSA concept:** This is an **optimization** to avoid redundant work.

## Relaxation Step:

```
for v in range(n):
    w = adj[u][v]
    if w != float('inf') and dist[v] > dist[u] + w:
        dist[v] = dist[u] + w
        prev[v] = u
        heapq.heappush(pq, (dist[v], v))
```

### *Line 1: Check all neighbors*

```
for v in range(n):
```

Loop through all possible nodes to find neighbors of u.

**DSA note:** With adjacency matrix, this is **O(n)** per node. With adjacency lists, it'd be **O(degree(u))** — more efficient.

### *Line 2: Get edge weight*

```
w = adj[u][v]
```

### *Line 3: Check if edge exists and if we can improve*

```
if w != float('inf') and dist[v] > dist[u] + w:
```

**Translation:**

1. "Is there a road from u to v?" (w != ∞)
2. "Is going through u faster than the current best route to v?" (dist[v] > dist[u] + w)

**Real-life analogy:**
Current best route to Karachi: 1300 km.
New route via Multan: 1200 km.
→ Update to 1200 km!

### *Lines 4-6: Update distance, predecessor, and queue*

```
dist[v] = dist[u] + w
prev[v] = u
heapq.heappush(pq, (dist[v], v))
```

1. Update shortest distance to v
2. Record that we came from u (for path reconstruction)
3. Add v to queue to explore its neighbors later

# PART 5: Path Reconstruction

```
def reconstruct_path(prev, start, end):
    path = []
    v = end
    while v != -1:
        path.append(v)
        v = prev[v]
    path.reverse()
    return path
```

**What this does:**
Follow the breadcrumbs backward from destination to start, then reverse.

**Real-life analogy:**
You're at Karachi and want to know how you got there:

1. Karachi ← Multan
2. Multan ← Lahore
3. Reverse: Lahore → Multan → Karachi

# VIVA QUESTIONS & ANSWERS

## Q1: Why use Dijkstra instead of BFS?

**A:** BFS finds shortest path in **unweighted** graphs (all edges = 1). Our graph has **different distances** between planets, so we need Dijkstra to handle weights.

## Q2: What if there are negative weights?

**A:** Dijkstra fails with negative weights. Use **Bellman-Ford** instead. But our distances are physical (always positive), so Dijkstra is correct.

## Q3: What's the time complexity?

**A:** With adjacency matrix: **$O(V^2)$**. With adjacency list + heap: **$O((V+E) \log V)$**. For our small graph (22 nodes), matrix is fine.

## Q4: Why rebuild the graph every frame?

**A:** Because planets are **moving**! Distances change, so edges change. We need a **dynamic graph** snapshot

# Graph Formation & Adjacency Matrix: The Complete Breakdown

Excellent questions, Aqi! Let's **dissect exactly** how this space graph is built and why the adjacency matrix changes from infinite values.

---

## 1. What Are The Vertices? (Counting the 24+ Things)

Let's trace through your **initialization code**:

```
# Initialize space system
bh = BH(W / 2, H / 2)  # 1 black hole

pl = []  # Planets
for _ in range(5):
    d = random.randint(130, 280)
    a = random.uniform(0, 2 * math.pi)
    pl.append(P(bh, d, a))  # 5 planets

st = []  # Satellites
for _ in range(12):
    st.append(S(random.choice(pl)))  # 12 satellites

strs = [St() for _ in range(40)]  # 40 stars/meteoroids
```

**Total Space Objects: 58 objects!**

- 1 black hole
- 5 planets
- 12 satellites
- 40 stars

**BUT WAIT!** Not all are graph vertices...

---

## 2. Which Objects Become Graph Vertices?

Look at the **graph building function**:

```
def bg(pl, st, bh):
    """
    Build adjacency matrix from planets, satellites, and black
hole.
    """
    an = pl + st + [bh]  # All nodes
    n = len(an)
    # ...
```

**Key Discovery:**

```
an = pl + st + [bh]
# an = [Planet0, Planet1, ..., Planet4, Sat0, Sat1, ..., Sat11,
BlackHole]
# Total vertices: 5 + 12 + 1 = 18 vertices
```

**Stars (meteoroids) are NOT included in the graph!** They're just visual decorations.

---

## 3. Vertex Indexing System

```
Vertex Index Map:
```

| Index Range | Object Type | Count |
|---|---|---|
| 0-4 | Planets (pl) | 5 |
| 5-16 | Satellites (st) | 12 |
| 17 | Black Hole (bh) | 1 |

```
Example:
an[0] = Planet 0
an[3] = Planet 3
an[5] = Satellite 0
an[10] = Satellite 5
an[17] = Black Hole
```

---

## 4. How The Adjacency Matrix Is Built (Step-by-Step)

### Step A: Initialize with Infinity

```
def bg(pl, st, bh):
    an = pl + st + [bh]  # 18 vertices
    n = len(an)  # n = 18
    am = [[float("inf")] * n for _ in range(n)]  # 18x18 matrix
```

**Initial State:**

```
# 18x18 matrix filled with infinity
am = [
    [inf, inf, inf, ..., inf],  # Row 0 (Planet 0)
    [inf, inf, inf, ..., inf],  # Row 1 (Planet 1)
    ...
    [inf, inf, inf, ..., inf],  # Row 17 (Black Hole)
]
```

**Why infinity?** It means "no direct connection" or "infinite cost to traverse."

## Step B: Connect Planets to Nearby Planets

```
# Connect planets within 350 pixels
for i in range(len(pl)):  # i = 0 to 4
    for j in range(i + 1, len(pl)):  # j = i+1 to 4
        dx = pl[i].x - pl[j].x
        dy = pl[i].y - pl[j].y
        d = math.sqrt(dx * dx + dy * dy)
        if d < 350:  # Distance threshold
            am[i][j] = d  # ← INFINITY REPLACED WITH DISTANCE!
            am[j][i] = d  # Undirected graph (both directions)
```

**Example Scenario:**

```
# Suppose we have 5 planets at these positions:
Planet 0: (400, 300)
Planet 1: (450, 320)  # Close to Planet 0
Planet 2: (600, 350)  # Far from Planet 0, close to Planet 1
Planet 3: (200, 400)  # Far from all
Planet 4: (420, 310)  # Close to Planet 0 and 1

# Distance calculations:
d(P0, P1) = sqrt((400-450)² + (300-320)²) = sqrt(2500 + 400) ≈
53.85
d(P0, P2) = sqrt((400-600)² + (300-350)²) = sqrt(40000 + 2500) ≈
206.16
d(P0, P3) = sqrt((400-200)² + (300-400)²) = sqrt(40000 + 10000) ≈
223.61
d(P0, P4) = sqrt((400-420)² + (300-310)²) = sqrt(400 + 100) ≈
22.36
d(P1, P2) = sqrt((450-600)² + (320-350)²) = sqrt(22500 + 900) ≈
152.97
```

**Matrix Updates (Row 0-4 only shown):**

```
# After planet-to-planet connections:
       P0     P1     P2     P3     P4    S0...S11  BH
P0  [  0,   53.8,  206.2,  inf,   22.4,  inf..., inf]
P1  [53.8,    0,   153.0,  inf,   30.0,  inf..., inf]
P2  [206.2, 153.0,   0,    inf,   inf,   inf..., inf]
P3  [inf,   inf,   inf,     0,    inf,   inf..., inf]
P4  [22.4,  30.0,  inf,    inf,    0,    inf..., inf]
```

**Key Insight:** Only planets within **350 pixels** get connected. Planet 3 is isolated!

---

## Step C: Connect Satellites to Their Parent Planets

```
# Connect satellites to their parent planets
for si, sat in enumerate(st):  # si = 0 to 11
    ni = len(pl) + si  # Node index: 5 to 16
    pi = pl.index(sat.o)  # Parent planet index: 0 to 4
    d = sat.od  # Orbital distance (35-65 pixels)
    am[ni][pi] = d  # ← REPLACE INFINITY!
    am[pi][ni] = d  # Bidirectional
```

**Example:**

```
# Suppose satellites orbit like this:
Sat0 (index 5) orbits Planet0 (index 0) at distance 45
Sat1 (index 6) orbits Planet1 (index 1) at distance 52
Sat2 (index 7) orbits Planet0 (index 0) at distance 38
...
Sat11 (index 16) orbits Planet4 (index 4) at distance 60

# Matrix updates:
       P0    P1    P2    P3    P4    S0    S1    S2  ...  S11
BH
P0  [  0,   53.8, 206.2, inf,  22.4,  45,   inf,  38, ..., inf,
inf]
P1  [53.8,   0,  153.0, inf,  30.0,  inf,  52,   inf,..., inf,
inf]
S0  [ 45,   inf,  inf,  inf,  inf,   0,    inf,  inf,..., inf,
inf]
S1  [inf,   52,   inf,  inf,  inf,   inf,  0,    inf,..., inf,
inf]
S2  [ 38,   inf,  inf,  inf,  inf,   inf,  inf,  0,  ..., inf,
inf]
```

**Important:** Satellites ONLY connect to their parent planet, not to each other!

---

## Step D: Connect Black Hole to Nearby Planets

```
# Connect black hole to nearby planets
bi = len(an) - 1  # Black hole index = 17
for i, pt in enumerate(pl):  # i = 0 to 4
    dx = pt.x - bh.x
    dy = pt.y - bh.y
    d = math.sqrt(dx * dx + dy * dy)
    if d < 450:  # Distance threshold
        am[i][bi] = d  # ← REPLACE INFINITY!
        am[bi][i] = d
```

**Example:**

```
# Black hole is at center (500, 375)
# Planets orbit at distances 130-280 pixels from black hole

d(P0, BH) = 130  # Within 450
d(P1, BH) = 150  # Within 450
d(P2, BH) = 200  # Within 450
d(P3, BH) = 280  # Within 450
d(P4, BH) = 140  # Within 450

# All planets connect to black hole!

# Final matrix (simplified view):
       P0    P1    P2    P3    P4   S0-S11  BH
P0  [  0,   53.8, 206.2, inf,  22.4, ...,  130]
P1  [53.8,   0,  153.0, inf,  30.0, ...,  150]
P2  [206.2,153.0,  0,   inf,  inf,  ...,  200]
P3  [inf,  inf,  inf,    0,   inf,  ...,  280]
P4  [22.4, 30.0, inf,  inf,    0,   ...,  140]
BH  [ 130,  150,  200,  280,  140,  ...,   0]
```

---

## 5. Complete Adjacency Matrix Visualization

Here's the **full 18x18 matrix** structure:

```
# Legend:
# P0-P4: Planets (indices 0-4)
# S0-S11: Satellites (indices 5-16)
# BH: Black Hole (index 17)

Adjacency Matrix (18x18):
     P0   P1   P2   P3   P4   S0   S1   S2  ...  S11  BH
P0 [  0,  53, 206, inf,  22,  45, inf,  38, ..., inf, 130]
P1 [ 53,   0, 153, inf,  30, inf,  52, inf, ..., inf, 150]
P2 [206, 153,   0, inf, inf, inf, inf,  60, ..., inf, 200]
P3 [inf, inf, inf,   0, inf, inf, inf, inf, ...,  55, 280]
P4 [ 22,  30, inf, inf,   0, inf, inf, inf, ...,  60, 140]
S0 [ 45, inf, inf, inf, inf,   0, inf, inf, ..., inf, inf]
S1 [inf,  52, inf, inf, inf, inf,   0, inf, ..., inf, inf]
S2 [ 38, inf,  60, inf, inf, inf, inf,   0, ..., inf, inf]
...
S11[inf, inf, inf,  55,  60, inf, inf, inf, ...,   0, inf]
BH [130, 150, 200, 280, 140, inf, inf, inf, ..., inf,   0]
```

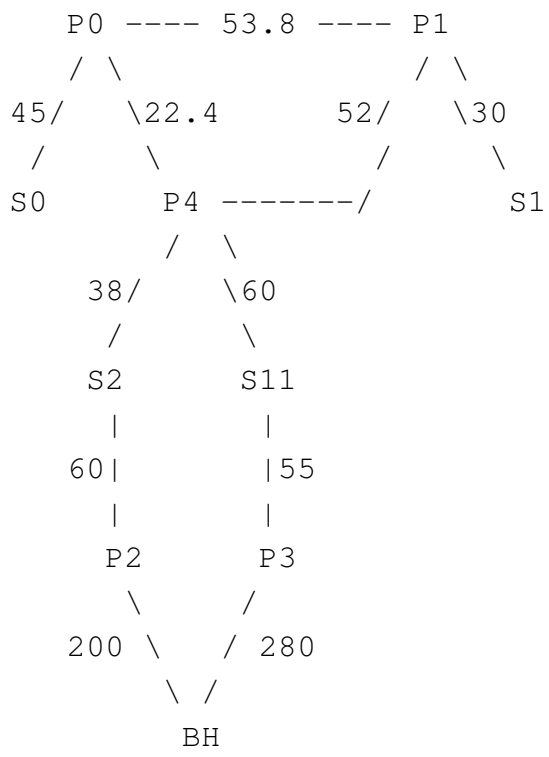
**Matrix Properties:**

- **Diagonal = 0** (distance from node to itself)
- **Symmetric** (undirected graph: am[i][j] = am[j][i])
- **Sparse** (mostly infinity - few connections)

---

## 6. Graph Visualization

Let me draw the **actual graph structure**:




**Connection Types:**

1st**Planet ↔ Planet**: Distance < 350 pixels

2nd **Satellite ↔ Parent Planet**: Orbital distance (35-65)

3rd **Planet ↔ Black Hole**: Distance < 450 pixels

---

## 7. How Infinity Values Change: The Transformation

```
# BEFORE graph building:
am[0][5] = inf  # No connection known

# DURING graph building (Step C):
# Sat0 orbits Planet0 at distance 45
ni = 5  # Satellite 0's index
pi = 0  # Planet 0's index
d = 45  # Orbital distance

am[ni][pi] = d  # ← inf becomes 45!
am[pi][ni] = d  # Symmetry

# AFTER:
am[0][5] = 45   # Connection established!
am[5][0] = 45
```

**The transformation happens EXACTLY when:**

1st**Condition is met** (distance < threshold, or parent-child relationship)

2nd **Distance is calculated**

3rd **Infinity is REPLACED with actual distance**

---

## 8. Why This Design?

### A. Infinity as "No Connection"

```
# When Dijkstra checks edges:
for v in range(nn):
    if am[u][v] < float("inf"):  # ← This check!
        # Only process if there's a connection
        nd = ds[u] + am[u][v]
```

**Infinity acts as a sentinel value** meaning "skip this, no edge exists."

### B. Dynamic Connections

```
# Planets move! So graph is rebuilt every frame:
for ev in pygame.event.get():
    if ev.type == pygame.MOUSEBUTTONDOWN:
        am, an = bg(pl, st, bh)  # ← REBUILD GRAPH!
```

**Graph topology changes** as planets orbit because distances change!

---

## 9. Memory Footprint

```
# 18 vertices → 18×18 = 324 cells
# Each cell = 8 bytes (float)
# Total: 324 × 8 = 2,592 bytes ≈ 2.5 KB

# Plus vertex objects:
# 5 planets × ~200 bytes = 1 KB
# 12 satellites × ~250 bytes = 3 KB
# 1 black hole × ~100 bytes = 0.1 KB
# Total: ~6.6 KB for entire graph system
```

**Very lightweight!** Modern RAM laughs at this.

---

## 10. Example Pathfinding Scenario

Let's trace a **real path**:

```
# User clicks Satellite 0 (index 5) and Black Hole (index 17
```